
SMART SURVEILLANCE & FACE RECOGNITION SYSTEM

System Architecture & Technical Implementation Manual

Document Details:

Title: Technical Architecture and Implementation Guide

Author: Technical Architecture Team

Date: July 28, 2026

Status: Final / Released

Target System: Python / OpenCV / Flask / Twilio Security Deployment

1. Executive Summary

The **Smart Surveillance and Face Recognition System** is a lightweight, edge-compatible security solution designed to monitor video streams in real-time, detect and identify faces, and automatically coordinate alert notifications upon identifying unauthorized personnel. The system operates on standard consumer webcams or IP-based CCTV cameras using the Real-Time Streaming Protocol (RTSP).

By integrating computer vision models with web server frameworks and cloud notification APIs, the application bridges the gap between legacy analog security hardware and modern intelligent automation. Key capabilities include real-time facial encoding extraction, alternate frame processing optimizations, a dual-layer local database cache, dynamic configuration reload, and asynchronous notification delivery.

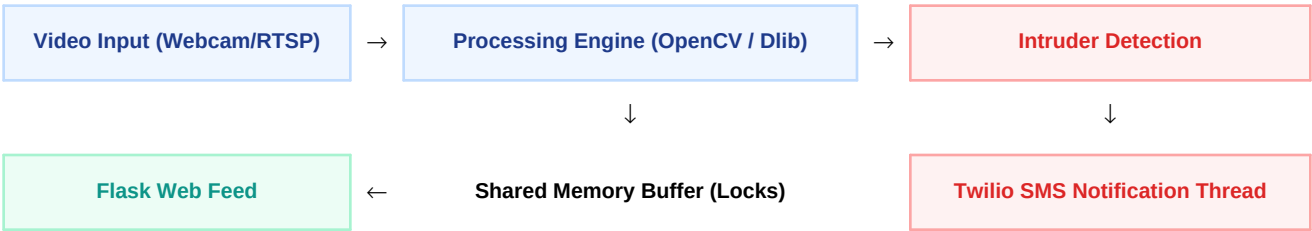
1.1 Document Scope

This document details the system's software architecture, multi-threaded concurrency model, component implementation mechanisms, performance optimization strategies, database design, API schemas, and deployment instructions.

2. System Architecture Overview

The software architecture follows a modular, layer-oriented structure separated into three primary sub-systems:

- **Core Computer Vision & Processing Engine:** Manages hardware stream capture, frame image preprocessing (scaling and color conversion), face localization, feature encoding extraction, and comparison vector matching.
- **Backend Server & APIs (Flask):** Powers the central dashboard interface, streams real-time annotated security video feeds, maintains the configuration states, and exposes endpoint routing for administrative interactions.
- **Alert & Notification Subsystem:** Dispatches SMS security notifications via Twilio, executing operations in secondary thread wrappers to isolate network latency and ensure zero drops in the primary video processing pipeline.



3. Concurrency Model & Multithreading

To achieve a smooth user experience, the system isolates high-latency and high-CPU tasks into separate threads. This guarantees that the Flask web dashboard remains responsive, camera feeds maintain a high frame rate, and SMS notifications do not block the video processing pipeline.

3.1 Thread Segmentation

The system coordinates three distinct classes of threads during runtime:

1. **Main Web Server Thread:** Serves Flask dashboard layouts, processes incoming REST API requests, and executes yielding of camera frame streams to HTTP clients.
2. **Camera Streaming & Processing Worker (Daemon Thread):** Executes in an infinite loop while the system is active. It connects to the input source, fetches frames, processes face recognition, runs alert checks, and draws HUD annotations.
3. **Twilio Notification Dispatcher (Asynchronous Threads):** Spun up on-demand as background daemon threads when a security alert is triggered. This offloads the network-bound latency of Twilio's HTTP REST client from the camera processing loop.

3.2 Synchronization & Thread Safety

Because multiple threads share the same variables, synchronization primitives are implemented to prevent race conditions:

`frame\_lock` (threading.Lock): Protects the `latest_processed_frame` variable. The background worker writes to this variable after annotating a frame, while the Flask MJPEG stream generator reads from it to yield data to the browser client.

`db\_lock` (threading.Lock): Protects the memory caches for `known_face_encodings` and `known_face_names`. When an administrator uploads a new authorized face via the Flask dashboard, it writes the photo to disk and calls `load_known_faces()`, updating the encoding arrays. The lock prevents the background worker from reading these arrays during a write cycle, avoiding memory corruption or runtime exceptions.

3.3 Alternate Frame & Downscaling Optimization

Calculating 128-dimensional face vectors from full-resolution high-definition images is extremely resource-intensive. The system incorporates two algorithmic optimizations inside the processing loop to reduce CPU load:

- **Resolution Downscaling (0.25x):** Before processing face detection, the frame is scaled down to 25% of its original resolution (`cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)`). Once face coordinates are identified, they are multiplied by 4 to scale annotations back up to the native high-definition display frame.
- **Frame Interleaving (Alternate Frame Processing):** Facial detection is executed only on every alternate frame (`process_this_frame = not process_this_frame`). On skipped frames, the face locations from the previous run are reused. This reduces computational requirements by 50% without compromising tracking quality.

4. Core Component Implementation Details

The codebase is divided into modular packages and helper files. Below is an in-depth breakdown of the main files:

4.1 System Entry Point & Dashboard Server: `app.py`

The app.py module initializes the Flask framework and manages the camera processing thread lifecycle. Key backend methods include:

- `background_camera_worker()`: Configures the OpenCV camera, executes the primary capture loop, handles hot-swapping cameras, and controls the alert cooldown.
- `load_known_faces()`: Scans `known_faces/` directory, parses file metadata, calculates the 128-D vectors, and loads them into memory.
- `yield_frames()`: Generator yielding processed JPEG bytes formatted as standard multipart MJPEG boundaries (`--frame\r\nContent-Type: image/jpeg\r\n\r\n[BYTES]\r\n`).

4.2 CLI Runner: `main.py`

A standalone script containing identical capture and facial recognition logic, but configured to write directly to a local window using `cv2.imshow()` instead of streaming to a Flask web server. Useful for headless test deployments or running diagnostics on local screens.

4.3 Configuration Module: `config/settings.py`

Manages variables stored inside the `.env` file. Exposes properties like `VIDEO_SOURCE`, `TOLERANCE`, and `COOLDOWN_SECONDS`. Includes a critical method:

`save_settings(updates)`: *Re-writes the physical `.env` file dynamically when updated values are posted through the admin panel. Simultaneously updates in-memory variables. Because of this hot-reload design, changes to tolerance, cooldown times, and video sources take effect instantly without requiring a server reboot.*

4.4 Alert Dispatcher: `alerts/twilio\_notifier.py`

Interfaces with the Twilio Python library. Instantiates a client using the configuration SID and Token. If credentials are using placeholders, the notifier gracefully cascades into a simulated terminal log print, preventing the software from crashing. Actual alerts run in daemonized thread loops:

```
thread = threading.Thread(target=self._send_sms_thread, args=(message_body,))
```

5. Data Dictionary & Storage Design

The system operates with a hybrid approach of disk-based file storage and in-memory caches to maintain speed.

5.1 Storage Directories

- **`known\_faces/`**: Flat database folder containing images of authorized personnel. The file's basename (excluding extension) acts as the lookup index (e.g. praveen.jpg yields the name 'Praveen' in the database). Supported formats: JPG, JPEG, PNG.
- **`captured/`**: Contains screenshots of intrusion events. Standard nomenclature: captured_unknown_YYYY-MM-DD_HH-MM-SS.jpg. Enables history logs and chronological timeline tracking.

5.2 User Enrollment Validation Flow

When an administrator enrolls a face, the system applies a strict safety gate: before adding the image permanently, the server loads it and runs facial landmark checks. If no face is found (or if multiple faces are found), the image is rejected and deleted, returning an API error. This guarantees the in-memory cache remains uncorrupted.

6. API Endpoints Reference

Method	Endpoint	Description	Payload / Query
GET	/	Serves the administrator dashboard template.	None
GET	/video_feed	Streams MJPEG processed feed stream.	None
GET	/api/settings	Fetches current system configurations.	None
POST	/api/settings	Updates configurations dynamically.	JSON config map
POST	/api/upload	Uploads photo and registers a known face.	FormData: name, photo
GET	/api/captured	Retrieves historical lists of unknown intruders.	None
GET	/captured_photos/<file>	Serves historical JPEG files from disk.	Filename path parameter

7. Technical Performance Parameters

Face Tolerance: The threshold value controls the Euclidean distance strictly. Lower values are stricter. A default of 0.6 is balanced. A setting below 0.5 decreases false positives but increases false negatives.

Alert Cooldown: A timer preventing Twilio credit drain. If set to 300 (5 minutes), the system will write intruder screenshots locally on every frame detection, but will only send 1 SMS alert every 5 minutes.

8. Summary of Benefits & Architecture Strengths

- **High Pipeline Responsiveness:** Network latency from SMS calls is isolated via thread-offloading, preventing video freeze.
- **Zero-Reboot Dynamic Updates:** The configurations can be updated directly from the UI dashboard, instantly triggering updates on the camera thread without halting processes.
- **Low Resource Consumption:** Downscaling and frame-skipping optimizations allow execution on low-cost devices like Raspberry Pi 4/5 or standard office desktop setups.